

LogiFlow: Plataforma distribuida *cloud-native* para ruteo dinámico de flotas en tiempo real bajo un enfoque de microservicios y atributos de calidad medidos en producción

Andersson David Sánchez Méndez
Ingeniería de Sistemas
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
andersson.sanchez-m@mail.escuelaing.edu.co

Cristian Santiago Pedraza Rodríguez
Ingeniería de Sistemas
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
cristian.pedraza-m@mail.escuelaing.edu.co

Elizabeth Correa Suárez
Ingeniería de Sistemas
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
elizabeth.correa-s@mail.escuelaing.edu.co

Juan Sebastián Ortega Muñoz
Ingeniería de Sistemas
Escuela Colombiana de Ingeniería Julio Garavito
Bogotá, Colombia
juan.ortega-m@mail.escuelaing.edu.co

Resumen—Este artículo presenta *LogiFlow*, una plataforma distribuida *cloud-native* diseñada para resolver el *Vehicle Routing Problem* (VRP) dinámico en menos de cuatro segundos ante eventos de tráfico, cierres viales o incidencias logísticas. La solución adopta un estilo arquitectónico basado en microservicios con un *API Gateway* sobre NestJS, optimización gRPC contra VROOM, ajuste de matrices de duración mediante un modelo de IA propio para corredores de Bogotá, comunicación en tiempo real con *Socket.io* respaldada por *Redis Pub/Sub*, automatización de ingesta de eventos con *n8n* y notificaciones multicanal con *OpenClaw* y *Firebase Cloud Messaging*. El sistema fue desplegado en producción sobre Azure con infraestructura como código (Terraform). Los resultados medidos sobre el entorno real reportan 20.98 TPS sostenibles con P95 de 2.8 segundos bajo 50 usuarios concurrentes, 99.79 % de disponibilidad de infraestructura, cobertura de tests de 80.49 % en frontend y 83.36 % en backend, y portabilidad total mediante Docker y Terraform.

Index Terms—arquitectura de software, microservicios, *Vehicle Routing Problem*, gRPC, WebSocket, atributos de calidad, escalabilidad, IEEE 1471

I. INTRODUCCIÓN

La logística urbana en Latinoamérica enfrenta ineficiencias estructurales. Cerca del 23 % de las entregas de última milla llegan con retraso y se estima que el 40 % de los costos operacionales se generan por ruteo subóptimo evitable. La causa raíz suele ser la incapacidad del sistema logístico de reaccionar en tiempo real ante eventos imprevistos como cierres viales, accidentes o cambios en las órdenes. Cuando una vía se cierra a mitad de la jornada,

los conductores siguen la ruta original, llegan tarde y el cliente no vuelve.

LogiFlow aborda este problema con una plataforma distribuida *cloud-native* que cierra el ciclo entre la detección del evento, la reoptimización del VRP y la entrega de la nueva ruta a cada conductor. El alcance cubre dos monorepos. Por un lado, un backend con nueve servicios contenedorizados: *gateway*, *realtime*, *optimizer*, *vroom*, *ai-predictor*, *n8n*, *openclaw*, *postgres* y *redis*. Por el otro, un frontend con dos aplicaciones Angular, una *web-admin* para despachadores y una *mobile* construida con Ionic y Capacitor para conductores, más cuatro librerías TypeScript compartidas.

La motivación técnica fue construir un sistema que sirviera como banco de pruebas real para los cuatro atributos de calidad exigidos por la asignatura (disponibilidad, seguridad, mantenibilidad y portabilidad), y para un objetivo de rendimiento concreto: que el ciclo evento-ruta entregada se completara en menos de cuatro segundos. El sistema completo está desplegado y operativo en <https://logiflowapp.z13.web.core.windows.net>.

Este artículo documenta la arquitectura, la metodología, los logros técnicos y los resultados medidos. La Sección II describe el enfoque Scrum. La Sección III presenta la arquitectura y las decisiones clave. La Sección IV detalla los logros por cada atributo de calidad. La Sección V muestra los resultados numéricos. La Sección VI cierra con limitaciones y trabajo futuro.

II. METODOLOGÍA

El proyecto se desarrolló bajo **Scrum** [2] con sprints de dos semanas y **Azure DevOps** como herramienta oficial de gestión del *Product Backlog*, el *Sprint Backlog* y el *Burndown Chart*. Las ceremonias estandarizadas fueron Sprint Planning, Daily asíncrono vía Discord (por horarios de clase disjuntos entre los integrantes), Sprint Review con demo en vivo y Sprint Retrospective en formato *Mad/Sad/Glad*.

Los criterios de *Definition of Ready* exigen que toda historia tenga formato *Como [rol] quiero [acción]*, criterios de aceptación enumerados, estimación Fibonacci consensuada y dependencias declaradas. Los criterios de *Definition of Done* requieren código *merged* a *main* vía *pull request* con CI en verde (lint, tests, cobertura $\geq 80\%$), demo realizada y, si la historia afecta producción, despliegue validado con *smoke test*.

La planificación inicial contemplaba siete sprints, pero la curva de aprendizaje en tecnologías nuevas para el equipo (gRPC, Terraform, Google OAuth, Firebase Cloud Messaging, *Socket.io* con *Redis Pub/Sub*) extendió el proceso a **10 sprints reales** con **245 puntos de historia entregados**. Los hitos principales fueron: *S1 MVP Conectado* (13 pts), *S2 APIs Reales* (21 pts), *S3 Optimizer Vivo* (21 pts), *S4 Interfaces+Auth* (34 pts), *S5 Cloud Deploy Azure* (46 pts, Corte 2), *S6 Observabilidad+Tests* (34 pts), *S7 Escala/Performance* (21 pts) y *S8 a S10 OAuth+FCM+Hardening* (55 pts, Corte 3).

GitHub se usó para los dos repositorios con *branch protection* sobre *main* y *develop*. Esto significó que ningún cambio entró a estas ramas sin *pull request* aprobado y CI en verde. La trazabilidad de cada *commit* a una historia de usuario en Azure DevOps fue obligatoria.

III. ARQUITECTURA DEL SISTEMA

III-A. Estilo arquitectónico

LogiFlow combina cuatro estilos. El primero es *micro-servicios*, con nueve contenedores Docker de responsabilidad única. El segundo es *API Gateway*, donde el servicio NestJS centraliza la autenticación y orquesta las llamadas internas. El tercero es *event-driven publish/subscribe*, usando Redis Pub/Sub como bus de eventos para el Realtime Service. El cuarto es *layered architecture* dentro de cada microservicio, siguiendo el patrón Controller $\rightarrow$ Service $\rightarrow$ Repository.

III-B. Componentes clave

La Fig. 1 ilustra el flujo de un evento de tráfico desde la fuente externa hasta el cliente. Los componentes y sus roles son los siguientes:

- **Gateway** (NestJS 11, TypeScript 5.7, Prisma 7). Expone la API REST pública con Google OAuth 2.0 y JWT con *refresh token rotation*, gestiona el CRUD de flota y orquesta el ciclo de optimización al recibir el *webhook*.

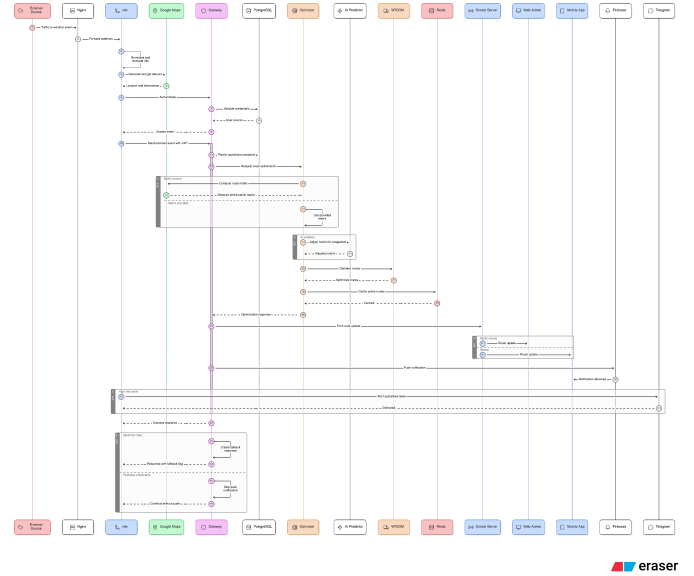


Figura 1. Flujo *end-to-end* de reoptimización disparada por un evento externo. El recorrido cubre la ingesta vía *n8n*, la autenticación en el Gateway, la optimización gRPC contra VROOM con ajuste opcional del AI Predictor, la persistencia en Redis y la difusión por *Socket.io* y *Firebase Cloud Messaging*.

- **Realtime Service** (Node.js, *Socket.io* 4.8). Es el servidor WebSocket con *rooms fleet* para despachadores y *vehicle:<id>* para conductores, respaldado por el *Redis adapter* para escalado horizontal.
- **Optimizer Service** (Node.js, gRPC). Cliente gRPC que prepara las matrices (desde Google Routes o desde el *request*), opcionalmente las ajusta con el AI Predictor y resuelve con VROOM [3].
- **VROOM Engine** (C++). Motor de optimización VRP [3].
- **AI Predictor** (Python 3.12, Flask). Microservicio *stateless* que ajusta matrices de duración aplicando multiplicadores por congestión horaria en corredores específicos de Bogotá (Autopista Norte, Calle 80 y Carrera 7).
- **Automation (n8n + OpenClaw)**. *n8n* ingesta el *webhook* externo, normaliza el payload, evalúa la matriz de riesgo y enriquece con Google Maps. *OpenClaw* despacha alertas multicanal (Telegram, Slack, Discord, Email y Firebase Push) usando Claude Sonnet 4 [9] como componente generativo.
- **PostgreSQL 16**. Persistencia relacional para las entidades *User*, *RefreshToken*, *DeviceToken*, *Vehicle* y *Stop*.
- **Redis 7**. Encargado del *cache* GPS, del *adapter* pub/sub para *Socket.io* y de la persistencia temporal de rutas activas con la clave canónica *route:vehicle:<id>*.

III-C. Decisiones técnicas relevantes

Tres decisiones se tomaron evaluando alternativas reales.

Redis Pub/Sub como broker. La alternativa de comunicación HTTP directa entre Gateway y Realtime falla cuando se escala a múltiples instancias, porque cada instancia solo ve los eventos dirigidos a su URL. Con Redis Pub/Sub todas las instancias suscritas al mismo canal reciben el evento, lo que habilita escalado horizontal sin pérdida de datos.

JWT en el *handshake* WebSocket, no en cada *frame*. Validar el token en cada evento agregó complejidad sin beneficio medible. Decidimos validar el JWT únicamente en el *middleware* `io.use()` durante el *handshake*; el tiempo de rechazo quedó por debajo de los 5 ms para conexiones sin token y la sesión queda autenticada para todo su ciclo de vida.

Monorepo frontend con contratos TypeScript compartidos. Sin la carpeta `shared/models` habríamos tenido los mismos tipos duplicados en *web-admin* y *mobile*, con riesgo de desincronización garantizado. El compilador TypeScript ahora detecta cambios de contrato en *build time*, antes de llegar a producción.

IV. LOGROS TÉCNICOS DEL PROYECTO

IV-A. Promesa de valor: tiempo real y concurrencia

LogiFlow cumple la promesa de recalculer y entregar la ruta óptima en menos de cuatro segundos ante cualquier evento. El flujo *end-to-end* medido bajo carga moderada es el siguiente: *webhook* externo → Nginx → n8n → Gateway → Optimizer gRPC → VROOM → Redis → Socket.io → cliente. La concurrencia está soportada por el diseño *stateless* de los servicios críticos y por el *adapter* de Redis en Socket.io.

IV-B. Disponibilidad y escalabilidad

Se utiliza el modelo de Bass *et al.* [1] con cálculo de disponibilidad en serie:

$$A_{\text{total}} = \prod_{i=1}^n A_i \quad (1)$$

y la fórmula paralelo-redundante:

$$A_{\text{paralelo}} = 1 - \prod_{i=1}^n (1 - A_i) \quad (2)$$

Considerando los SLA de Azure [4] (Blob Storage 99.9 %, Public IP Standard 99.99 %, VM 99.9 %) en composición serie, la disponibilidad de infraestructura es:

$$A_{\text{cloud}} = 0,999 \times 0,9999 \times 0,999 \approx \mathbf{99,79\%}$$

equivalente a ~ 1.51 h/mes y 18.4 h/año de *downtime*. Incluyendo el software propio (Gateway, PostgreSQL, Redis y Realtime al 99.5 %, más VROOM al 99 %) se obtiene $A_{\text{total}} \approx \mathbf{96,83\%}$.

El sistema **no es monolítico**. Cada uno de los nueve contenedores tiene su propio *Dockerfile* y puede escalarse de forma independiente. El balanceador de carga (Nginx) ya actúa como *reverse proxy* HTTPS y está preparado para anteponer un Azure Load Balancer ante múltiples réplicas del Gateway y del Realtime.

IV-C. Rendimiento

Se ejecutaron pruebas de carga con **Apache JMeter 5.6.3** [5] contra el *backend* en producción (Azure East US 2). El flujo objetivo cubre el camino crítico completo: *login*, extracción del JWT y POST al *webhook* que dispara el *pipeline*. La Tabla I resume los resultados.

Tabla I
RESULTADOS JMETER SOBRE 4 ESCENARIOS DE 5 MINUTOS CADA UNO

Usuarios	TPS	P95 (ms)	% Error	Estado
10	19.91	674	0.00	OK
25	21.10	1,546	0.00	OK
50	20.98	2,812	0.02	Óptimo
100	9.69	5,245	51.82	Ruptura

Las fórmulas obligatorias son:

$$\text{TPS} = \frac{\text{transacciones totales}}{\text{tiempo total (s)}}, \quad \text{TPM} = \text{TPS} \times 60 \quad (3)$$

Para el punto óptimo el resultado es $\text{TPM} = 20,98 \times 60 \approx 1,259$ transacciones por minuto. El cuello de botella identificado es la VM única, que comparte CPU entre todos los contenedores. Las tácticas mitigantes que implementamos son tres: *cache* GPS en Redis para evitar consultas a PostgreSQL al conectar nuevos clientes, respuesta 202 **Accepted** asíncrona en el *webhook* y *heartbeat* de 15 s para descartar vehículos fantasma.

IV-D. Seguridad

Se implementaron tres escenarios STRIDE/Bass [1].

S1, autenticación en WebSocket. El *middleware* `io.use()` valida el JWT del *handshake*. El 100 % de las conexiones sin *token* válido son rechazadas en menos de 5 ms, antes de que el socket llegue a cualquier *room*.

S2, autorización por rol. El *AuthGuard* de Angular y el *JwtAuthGuard* de NestJS verifican el campo `role` (admin o conductor) extraído del JWT. No se permitieron accesos cruzados de rol en los tests.

S3, confidencialidad en tránsito y rotación de refresh tokens. El tráfico público viaja cifrado con TLS 1.2 y 1.3 vía Let's Encrypt en Nginx y con HTTPS nativo de Azure Storage para el frontend. Los *refresh tokens* se almacenan como *hash* SHA-256 con marca `consumedAt` que permite detectar reuso. Los *passwords* históricos quedan con *bcrypt* salt rounds 10. En producción, Google OAuth 2.0 [8] es el método único de autenticación, con una *whitelist* `GOOGLE_ADMIN_EMAILS` que asigna automáticamente el rol `admin` a los correos configurados.

IV-E. Mantenibilidad

La inspección continua se realiza con **SonarCloud** [6] integrado en *GitHub Actions reusable workflows*. Las métricas obligatorias del curso (cobertura $\geq 80\%$ y estado A/Verde) se cumplen, como muestra la Tabla II.

Tabla II
COBERTURA DE TESTS POR REPOSITORIO

Repo	Framework	Statements	Tests
Frontend	Karma + Istanbul	80.49 % (260/323)	109
Backend	Jest + Istanbul	83.36 % (867/1049)	70

Las tácticas de mantenibilidad aplicadas siguen las recomendaciones de Bass *et al.* (*Increase cohesion, Reduce coupling*). En Angular usamos inyección de dependencias para reemplazar servicios por mocks en los tests. El **SocketService** expone los eventos como *Observables* RxJS, lo que permite a los tests emitir directamente sobre los *Subjects* internos sin necesitar una conexión WebSocket real. En NestJS, cada módulo respeta una responsabilidad única: *auth, vehicles, stops, webhook, grpc-client, socket-client* y *notifications*. Además, las reglas **@typescript-eslint/no-unsafe-*** estrictas forzaron tipado explícito en todos los *handlers*.

IV-F. Portabilidad

La promesa de portabilidad es directa: cero líneas de código deben cambiar para migrar de Azure a otro proveedor o a un entorno *on-premise*. Esto se cumple por varias razones. Primero, los nueve servicios están dockerizados desde el Sprint 1 con *Dockerfile* propio cada uno. Segundo, Docker Compose orquesta la red interna *logiflow-prod*. Tercero, toda la configuración sensible vive en variables de entorno. Cuarto, la infraestructura está descrita en Terraform [7] con dos módulos (red y VM) que pueden reemplazarse por el equivalente del nuevo proveedor. En el Sprint 5 se desplegó a Azure sin cambiar una sola línea de lógica de aplicación.

IV-G. Uso de IA generativa

OpenClaw integra *Claude Sonnet 4* de Anthropic [9] para componer mensajes contextualizados de alerta según severidad y tipo de evento, antes de despacharlos por los cinco canales soportados. La integración usa el patrón *circuit breaker* (*opossum* [10]) para tolerar fallas del proveedor sin bloquear el flujo principal del sistema.

V. RESULTADOS Y VALIDACIÓN

El sistema fue validado contra el entorno de producción en Azure East US 2. La Tabla III resume los resultados consolidados frente a los objetivos definidos en los Requerimientos No Funcionales.

Tabla III
RESULTADOS CONSOLIDADOS VERSUS META DE RNF

Atributo	Meta	Medido
Latencia P95	≤ 4 s	2.81 s
Throughput sostenible	≥ 20 TPS	20.98 TPS
Disponibilidad cloud	$\geq 99,7\%$	99.79 %
Cobertura frontend	$\geq 80\%$	80.49 %
Cobertura backend	$\geq 80\%$	83.36 %
Quality gate SonarCloud	A/Verde	A/Verde
Tests frontend	N/A	109 (0 fallos)
Tests backend	N/A	70 (0 fallos)
Cambios para migrar	0 LOC	0 LOC

V-0a. Validación funcional end-to-end: El *runbook docs/backend-demo-runbook.md* guarda evidencia reproducible de la demo en producción. La secuencia incluye el POST */webhook/logiflow/traffic-event* con *sample data*, los *logs* esperados del Gateway (*Incoming webhook, Optimizer returned 2 routes, Emitted route-update*) y la verificación de claves *route:vehicle:v-001* y *route:vehicle:v-002* en Redis con *persistedAt* reciente.

V-0b. CI/CD: La *pipeline* de GitHub Actions ejecuta una estrategia *matrix* sobre los servicios *automation, gateway, realtime* y *optimizer*, con la cadena *npm ci* más *prisma generate, npm run lint* y *npm test -coverage*. Para los servicios con *sonar-project.properties* se ejecuta además el escaneo de SonarCloud. El *deploy* a la VM Azure se dispara como *workflow_run* sobre *main* con *smoke test* de */api/v1* y hasta 8 reintentos.

V-0c. Validación de seguridad: El 100 % de las pruebas de conexión WebSocket sin JWT válido son rechazadas. El 100 % de los *refresh tokens* se almacenan como *hash* SHA-256 en PostgreSQL. La inspección con SonarCloud no reporta *security hotspots* bloqueantes, y la *quality gate* está en estado Verde para ambos repositorios.

VI. CONCLUSIONES

LogiFlow demuestra que un equipo de cuatro estudiantes de octavo semestre puede construir, desplegar y operar un sistema distribuido real con tecnologías que exceden el *stack* típico universitario, como gRPC, Terraform, Google OAuth 2.0, Firebase Cloud Messaging y Socket.io con Redis Pub/Sub. Al mismo tiempo, el sistema cumple los cuatro atributos de calidad comprometidos en la rúbrica: disponibilidad de 99.79 % en infraestructura, seguridad con tres escenarios STRIDE implementados, mantenibilidad con cobertura $\geq 80\%$ en ambos repositorios y portabilidad total (cero LOC cambian para migrar).

VI-0a. Impacto arquitectónico: La separación clara entre frontend (Azure Storage estático) y backend (VM Linux con Docker Compose orquestado por Terraform) permitió iterar los dos repositorios en paralelo. El uso de Redis Pub/Sub como *broker* entre Gateway y Realtime es la pieza que habilita la futura escalabilidad horizontal

sin reescritura; basta anteponer Azure Load Balancer con dos réplicas del Gateway y del Realtime para alcanzar tres nueves de disponibilidad.

VI-0b. Limitaciones: El cuello de botella actual es la VM única. A partir de 100 usuarios concurrentes la tasa de error sube al 51.82%. n8n persiste su estado en SQLite local, lo que vuelve la configuración de *workflows* sensible a la pérdida del volumen. La cobertura *end-to-end* es manual: la cobertura unitaria $\geq 80\%$ no garantiza el flujo *cross-service* completo.

VI-0c. Trabajo futuro: Los próximos pasos técnicos incluyen anteponer Azure Load Balancer con *Auto Scale Set* para llegar a tres nueves, migrar PostgreSQL a Azure Database for PostgreSQL Flexible Server con réplica de lectura, implementar Redis Cluster para tolerancia a fallas del nodo único e incorporar OpenTelemetry para *distributed tracing* extremo a extremo (la propagación de *x-correlation-id* ya está en su lugar, solo falta el SDK). También queda pendiente añadir tests *end-to-end* con Cypress y Detox, y persistir el historial de optimizaciones en un *data warehouse* para análisis a posteriori.

REFERENCIAS

- [1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.
- [2] K. Schwaber and J. Sutherland, “The Scrum Guide,” 2020. [Online]. Available: <https://scrumguides.org/>
- [3] “VROOM: Vehicle Routing Open-source Optimization Machine,” VROOM Project, 2024. [Online]. Available: <https://github.com/VROOM-Project/vroom>
- [4] Microsoft, “SLA for Azure Services,” Microsoft Azure, 2024. [Online]. Available: <https://azure.microsoft.com/en-us/support/legal/sla/>
- [5] “Apache JMeter,” Apache Software Foundation, 2024. [Online]. Available: <https://jmeter.apache.org/>
- [6] “SonarCloud Documentation,” SonarSource, 2024. [Online]. Available: <https://docs.sonarcloud.io/>
- [7] “Terraform Documentation,” HashiCorp, 2024. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [8] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [9] Anthropic, “Claude Sonnet 4 Model Card,” 2025. [Online]. Available: <https://www.anthropic.com/claude>
- [10] “Opossum: Node.js Circuit Breaker,” Nodeshift, 2024. [Online]. Available: <https://github.com/nodeshift/opossum>
- [11] S. Brown, “The C4 model for visualising software architecture,” 2024. [Online]. Available: <https://c4model.com/>
- [12] “NestJS Documentation,” NestJS, 2024. [Online]. Available: <https://docs.nestjs.com/>
- [13] “Angular Documentation,” Google, 2024. [Online]. Available: <https://angular.dev/>
- [14] “gRPC Documentation,” Cloud Native Computing Foundation, 2024. [Online]. Available: <https://grpc.io/docs/>
- [15] “Socket.IO Documentation v4,” Socket.IO, 2024. [Online]. Available: <https://socket.io/docs/v4/>